

إطار ذكاء الإجراءات: منهجية هندسية متكاملة لتصنيف وتحليل وتفاعل الإجراءات البرمجية في بيئات الذكاء الاصطناعي الوكيل

الجذور التاريخية والأسس النظرية للإجراءات البرمجية

يعود التأسيس التاريخي لوحدة المنطق البرمجي القابلة للاستدعاء إلى فجر عصر الحوسبة العملي في جامعة كامبريدج، حيث تولى الذي يُعد أول حاسوب عملي ذي برنامج، EDSAC، موريس ويلكس إدارة المختبر الرياضي منذ عام 1945. وقاد ويلكس بناء حاسوب مخزن في العالم، وقد اكتملت صياغته التشغيلية في مايو 1949. وفي هذا السياق، وضع عالم الحاسوب ديفيد ويلر، أثناء تحضيره لأول (Closed) رسالة دكتوراه في علوم الحاسوب في التاريخ، حجر الأساس لهندسة البرمجيات عبر اختراع مفهوم البرنامج الفرعي المغلق (Wheeler Jump) "وما عُرف تاريخياً باسم "قفزة ويلر (Subroutine) (Wheeler Jump).

وقبل شيوع مكدرات العناوين العتادية، واجهت الأجهزة المبكرة تحدي الاقتتار إلى آلية مدمجة لحفظ عناوين الرجوع تلقائياً. لحل هذه عام 1945 استخدام مكدر لحفظ العناوين، غير أن ويلر صاغ حلاً برمجياً ACE المعضلة، اقترح آلان تورينج في مسودات تصميم آلة في سبتمبر (Initial Orders 2) "والمعروف باسم "الأوامر الابتدائية الثانية EDSAC مبتكراً حقن في نظام التشغيل الأولي لحاسوب ؛ حيث يشتمل كود الاستدعاء على أمر يقوم (Self-Modifying Code) 1949. اعتمدت قفزة ويلر على نمط برمجيات ذاتي التعديل ثم يقفز إلى البرنامج الفرعي الذي يقوم بدوره في خطوته الأولى بتعديل (Accumulator) بتحميل عنوان خطوة العودة في المرمك أمر القفز النهائي الخاص به ديناميكياً ليوجه التدفق العكسي إلى نقطة الاستئناف الصحيحة. وواجه كتاب البرامج الفرعية آنذاك معضلة (Initial) وهو ما تم تجاوزه بتطوير برنامج الإدخال الأولي، (Pseudo-orders) عدم معرفة العنوان الدقيق لربط الكود في الذاكرة لضبط مراجع الذاكرة ذاتياً أثناء التحميل من الأشرطة الورقية المثقوبة (Initial Input Routine).

؛ ففي أول كتاب برمجي منهجي صاغه ويلكس وويلر (Callback) تزامن مع ولادة البرامج الفرعية بزوغ مفهوم استدعاء الارتجاع المخصص لحل المعادلات التفاضلية المشتركة. تطلب هذا البرنامج توفير G1 وجيل عام 1951، وُصف البرنامج الفرعي الرياضي يمرره المستخدم لحساب قيمة المشتقة الأولى، بحيث يستدعيه البرنامج الرئيسي (Auxiliary Subroutine) برنامج فرعي مساعد أربع مرات في كل خطوة حسابية، مما يمثل أول تطبيق تاريخي لآلية استدعاء الارتجاع قبل ابتكار المقاطعات العتادية إلى نموذجين مختلفين (Procedures) والإجراءات (Functions) من منظور لغوي ونظري، تستند الأصول الرياضية للدوال الذي طوره ألونزو تشيرش في الثلاثينيات كتجريد للتحويلات (Lambda Calculus) للحوسبة. ترتبط الدوال مباشرة بحساب لامبدا الرياضية النقية الخالية من الحالات والآثار الجانبية، حيث تُعامل الدالة كصندوق أسود يرسم علاقة حتمية بين المدخلات والمخرجات. حيث يمثل الإجراء تسلسلاً من العمليات المنظمة، (Imperative Model) أما الإجراءات، فتنبثق من نموذج آلة تورينج الإمبراطوري لتعديل حالة الآلة والبيئة المحيطة بها. ويوضح الجدول التالي المعالم التاريخية والنظرية لنشوء هذه المفاهيم في لغات البرمجة الأولية:

(Theoretical & Practical Contribution)	(Originator & Year)	(Historical Milestone)
التأسيس النظري المبكر لفكرة حفظ عناوين الرجوع عتادياً	(آلان تورينج) 1945	ACE مفهوم مكدر العودة في آلة
تمكين نمطية البرمجيات وإعادة استخدام الكود برمجياً دون تعديل عتادي	(ديفيد ويلر) 1949	البرنامج الفرعي المغلق وقفزة ويلر
يقوم بتحويل (Assembler) أول مجمّع الرموز والحروف إلى بنية ثنائية وتعديل العناوين	(ديفيد ويلر) 1949	(Initial Orders) برنامج الأوامر الابتدائية
توثيق 88 برنامجاً فرعياً لحساب الدوال الرياضية وصياغة أول واجهة برمجة تطبيقات (API)	(ويلكس، ويلر، جيل) 1951	أول مكتبة برمجية وكتاب صياغة برمجية
تمرير برنامج فرعي مساعد كمعامل لبرنامج حل المعادلات التفاضلية	(ويلكس، ويلر، جيل) 1951	أول مفهوم لاستدعاء الارتجاع (Callback)

الاختلافات الدلالية والتحقق الأكاديمي عبر منطق هور

أدى تطور لغات البرمجة المعاصرة إلى دمج الدوال والإجراءات في مسمى واحد، حيث تُعامل الإجراءات في اللغات المستوحاة من لغة ومع ذلك، فإن إهمال الفروق الدلالية بينهما يحد من كفاءة التحليل الساكن، ويعوق عمليات التوازي. (Void) كدوال ترجع قيمة فارغة C ؛ حيث تمنع القواعد الصارمة PL/SQL البرمجي والتحقق الرياضي. تظهر هذه الفروق جلياً في بيئات قواعد البيانات البرمجية مثل بينما يُسمح باستدعاء الدوال شريطة خلوها من الآثار الجانبية والعمليات، (SELECT) استدعاء الإجراءات داخل عبارات الاستعلام وإدارة المعاملات التبادلية (Try-Catch) كما تدعم الإجراءات معالجة الاستثناءات عبر كتل المحاولة والقبض. (DML) التعديلية وهو ما يمتنع عن الدوال في البيئات القياسية. ويقارن الجدول التالي بين هذين المفهومين عبر معايير برمجية، (Transactions) ونظرية متعددة:

(Procedures) الإجراءات التنفيذية	(Pure Functions) الدوال الرياضية النقية	(Comparison Criterion) معيار المقارنة الدلالية
آلة تورينغ والأنظمة الإمبراطورية	حساب لامبدا ونظرية الفئات	المرجعية الرياضية والتأسيس
خيار، ويمكن إرجاع مخرجات متعددة عبر معاملات (OUT)	إلزامية إرجاع قيمة مخرجة مفردة ومحددة دائماً	إرجاع القيم والمخرجات
أساسية؛ تُستدعى لتعديل الحالة والقيام بعمليات الإدخال والإخراج	ممنوعة؛ لا تغير متغيرات البيئة الخارجية	الآثار الجانبية وتعديل الحالة
تشكل عبارة برمجية مستقلة (Stand-alone Statement)	تدرج مباشرة كجزء من تعبير رياضي أو منطقي	الاستدعاء ضمن التعبيرات
تدعم معالجة الأخطاء وإدارة الصفقات والالتزام والتراجع	لا تدعم إدارة الصفقات وكتل معالجة الأخطاء السلسلة	إدارة المعاملات التبادلية والكتل
تتطلب تتبع مسارات الحالة والتحول عبر منطق هور	سهولة الإثبات والتحقق لوضوح علاقة المدخل بالمخرج	قابلية التحليل والتحقق الصارم

الصياغة الرياضية الأمثل (Hoare Logic) لتأسيس قاعدة برمجية متينة للتحقق الأكاديمي من سلوك الإجراءات، يوفر منطق هور للتعبير عن انتقال الحالة البرمجية. يركز هذا المنطق على ثلاثية هور:

$$\{P\} C \{Q\}$$

قبل التنفيذ، وانتهى الإجراء من العمل بنجاح، فإن P ؛ فإذا تم استيفاء الشرط المسبق C تصف هذه الثلاثية السلوك الديناميكي للإجراء لربط هذا الإطار الرياضي بالترجمة البرمجية العملية، تنقسم الصياغة المنطقية Q. الحالة النهائية للنظام تلبى بالضرورة الشرط اللاحق (Assignment Axiom): إلى مدرستين رئيسيتين لتطبيق بديهية الإسناد:

- **بديهية فلويد للإسناد الأمامي:** تركز على دفع الحالة إلى الأمام باستخدام التقدير الوجودي للمتغيرات السابقة، وصيغتها:

$$\vdash \{P\} V := E \{ \exists v. (V = E[v / V] \wedge P[v / V]) \}$$

- **بديهية هور للإسناد العكسي:** تركز على إرجاع الشرط اللاحق إلى نقطة البداية عبر استبدال تعبير الإسناد مباشرة، وصيغتها:

$$\vdash \{Q[E / V]\} V := E \{Q\}$$

مما يجعلها المفضلة في أنظمة التحقق البرمجي التلقائي لقدرتها على (exists) تتميز صيغة هور العكسية بخلوها من المكتمل الوجودي (Weakest) تجنب تراكم المتغيرات الوهمية أثناء التحليل التراجعي. وفي هذا الإطار، يُعرف الشرط المسبق الأضعف

إلى الشرط المسبق الأساسي والأكثر شمولاً لضمان صحة Q بأنه الأداة الرياضية التي تحول الشرط اللاحق (Precondition - wp) الإجراءات:

$$wp(V := E, Q) = Q[E / V]$$

التحول الأمامي للحالة، ويُعبر عنه بالصيغة الوجودية (Strongest Postcondition - sp) بينما يمثل الشرط اللاحق الأقوى:

$$sp(V := E, P) = \exists v. (V = E[v / V] \wedge P[v / V])$$

تسمح بربط خطوات (Composition Rules) لتأليف إثباتات برمجيات كاملة، يعتمد منطق هور على قواعد استدلال تركيبية:

الإجراءات المتسلسلة والشرطية والحلقات التكرارية بشكل صارم:

$$\frac{\{P\} C_1 \{R\} \quad \{R\} C_2 \{Q\}}{\{P\} C_1; C_2 \{Q\}} \quad (\text{Composition Rule}) \quad \frac{\{P \wedge B\} C_1 \{Q\} \quad \{P \wedge \neg B\} C_2 \{Q\}}{\{P\} [C_1; C_2] \{Q\}} \quad (\text{if } B \text{ then } C_1 \text{ else } C_2 \{Q\})$$

$$\frac{\{P \wedge B\} C \{P\}}{\{P\} \{ \text{while } B \text{ do } C \} \{P \wedge \neg B\}} \quad (\text{Conditional Rule}) \quad \frac{\{P \wedge B\} C \{P\}}{\{P\} \{ \text{while } B \text{ do } C \} \{P \wedge \neg B\}} \quad (\text{While Rule})$$

يظل صحيحاً قبل وبعد كل دورة (P - Loop Invariant) تحديد متغير ثابت للحلقة (While Rule) تتطلب قاعدة الحلقة التكرارية (Well-founded Relation) يجب دمج دالة رتبة تنازلية، (Total Correctness) تكرار، وفي حال رغبة المهندس في إثبات الصحة الكلية لضمان حتمية إنهاء الحلقة وتقادي التكرار اللانهائي (Relation).

التصنيف متعدد الأبعاد للإجراءات وهندسة الأنطولوجيا البرمجية

لتشييد إطار هندسي متكامل لذكاء الإجراءات، يجب صياغة نظام تصنيفي منهجي متعدد الأبعاد يتجاوز الهياكل المسطحة. ولضمان تماسك هذا التصنيف العلمي، يتعين تطبيق مبادئ بيدفورد السبعة لتصميم التصنيفات والأنطولوجيات لضمان جودة الهيكل وعزل الفئات

- بقاء القواعد والمعايير المعتمدة لتقسيم الفئات ثابتة عبر جميع مستويات الهيكل الهرمي: **(Consistency) الاتساق**
- وجوب اشتقاق معايير الفئات الفرعية مباشرة من خصائص الفئة الأبوية العليا لضمان دقة: **(Affinity) التقارب والقراءة** التوريث الدلالي.
- اشتراط وجود فئتين فرعيتين متميزتين على الأقل عند تجزئة أي مستوى تصنيفي: **(Differentiation) التمايز والتنوع** لضمان التعددية.
- خلو الفئات المتوازية من التداخل الدلالي، بحيث يُفرز الإجراء في فئة واحدة فريدة: **(Exclusiveness) الحصرية المتبادلة** دون لبس.
- إمكانية التحقق الإثباتي العملي من انتماء أي عنصر للفئة المحددة بناءً على: **(Ascertainability) اليقين والتحقق** خصائصه الظاهرة.
- قدرة البناء التصنيفي على استيعاب المصطلحات والتقنيات المعاصرة والناشئة دون: **(Currency) الحداثة والمعاصرة** الحاجة لإعادة هيكلته.
- استيعاب التصنيف لكافة العناصر والمتغيرات الموجودة في مجال الدراسة: **(Exhaustiveness) الشمول والاستقصاء** لضمان التغطية الكاملة.

يتكامل هذا الإطار التصنيفي مع المعايير الدولية والأنطولوجيات البرمجية المعتمدة، مثل تصنيف مواصفات ميزات الأدوات البرمجية يقسم هذا المعيار ميزات الأدوات إلى مستويات (NBS Special Publication 500-74) الصادر عن المكتب الوطني للمعايير تليها مرحلة الوظيفة، (Control) وفئة التحكم (Subject) المكونة من الفئة الموضوعية (Input) هرمية، تبدأ من مرحلة المدخلات والتحسين T3 وإضافة المستشعرات T1 مثل التنسيق (Transformation) المشتملة على عمليات التحويل (Function) والتحويل وانتهاءً، (Dynamic Analysis) والديناميكية (Static Analysis) والتحليلات الساكنة، T6 والترجمة T5 وإعادة الهيكلة T4 والموجهة للمستخدم أو الآلة (Output) بمرحلة المخرجات.

لفرز الأنشطة إلى مهام مجال حقيقية ترتبط ببيئة العمل (UFuRT) كما يستعير الإطار أدوات التحليل الموحد لسهولة الاستخدام ولتأصيل هذه (Overhead Functions) ومهام هامشية إضافية ترتبط بنظام التشغيل وإدارة الواجهات (Domain Functions) والأنطولوجيا المرجعية لاختبار البرمجيات (SP-OPL) المفاهيم معرفياً، تُربط هذه التصنيفات مع أنطولوجيا العمليات البرمجية يهدف هذا الدمج إلى إنشاء نموذج موحد يربط أنشطة دورة حياة (UFO) المؤصلة في الأنطولوجيا التأسيسية الموحدة (ROoST) البرمجيات بالإجراءات والأساليب والتقنيات المناسبة لتحقيق الأهداف التنظيمية. يوضح الجدول التالي توزيع الأبعاد والمستويات التصنيفية وفق هذه الأطر المدمجة:

دور البعد في هندسة البرمجيات (Engineering Application)	فئات التصنيف الرئيسية (Primary Subclasses)	المعيار الفني المعتمد (Reference Standard)	البعد التصنيفي الهيكلي (Taxonomic Dimension)
تصنيف ميزات وقدرات الأدوات البرمجية لتحديد مستوى كفاءتها وملاءمتها للمهام المطلوبة	مدخلات (موضوع، تحكم)، وظائف تشغيلية (تحويل، تحليل ساكن، تحليل ديناميكي)، مخرجات (آلة، مستخدم)	NBS SP 500-74	هيكل الميزات الوظيفية
تصفية العمليات وفصل الجهد التشغيلي للنظام عن متطلبات جودة منطق الأعمال الفعلي	وظائف المجال الأساسية ووظائف الجهد (Domain)، الهامشي الإضافي (Overhead)	الموحد UFuRT إطار	دلالة المهام والتشغيل
تأسيس مفردات مشتركة وتأصيل دلالي لربط الإجراءات بأهداف وإجراءات جودة المؤسسة	عمليات برمجية معيارية (Standard)، عمليات مكيفة (Tailored) ومخصصة	SP-OPL / UFO	نمط البنية الأنطولوجية

دور البعد في هندسة البرمجيات (Engineering Application)	فئات التصنيف الرئيسية (Primary Subclasses)	المعيار الفني المعتمد (Reference Standard)	البعد التصنيفي الهيكلي (Taxonomic Dimension)
تزويد المطورين والكلاء بنماذج إرشادية وتدفقات عمل محددة لتسهيل تنفيذ المهام المعقدة	برمجية تفصيلية (Methods)، أساليب منهجية (Techniques)، تقنيات فنية قوالب مستندات وثائق (Templates)، أدلة الأدوات (Tool Mentors)	البرمجية ROoST أنطولوجيا	طبيعة الممارسة والتوثيق

معمارية تفاعل الإجراءات في بيئات الذكاء الاصطناعي الوكيلية و بروتوكول

MCP

شهدت معمارية الأنظمة الوكيلية تحولاً جوهرياً بالانتقال من مجرد المحادثة الخطية إلى تبني حلقات التفاعل المستمر المتمثلة في دورة وضمن هذا السياق، يبرز بروتوكول سياق النموذج (Observe-Think-Act-Reflect) (O-T-A-R) الملاحظة والتفكير والعمل والارتجاع كمعيار مفتوح وموحد لتنظيم قنوات الاتصال بين العوامل الذكية ومصادر المعرفة (Model Context Protocol - MCP) والأدوات الخارجية. يسهم هذا التوحيد في تدريب النماذج وضبطها بدقة للتعرف على مخططات الأدوات ووسيطاتها وصياغتها وتفعيلها بأقل نسبة خطأ.

تطبيق الذكاء (Host) في بيئة عميل وخادم؛ حيث يمثل المضيف (Stateless) على تنظيم تواصل ولاولائي MCP تتأسس معمارية بينما JSON-RPC 2.0 إدارة الجلسات وتوجيه طلبات الاتصال المستندة إلى ترميز (Client) الاصطناعي الرئيسي، ويقود العميل كعملية معزولة تقدم ثلاث قدرات أساسية MCP يقبع خادم

- دوال وإجراءات تنفيذية يكتشفها النموذج اللغوي ويستدعيها ديناميكياً مع تمرير مدخلات مصدقة بالكامل: (Tools) الأدوات وتلقي استجابات مهيكلة.
- ميزة، لتمكين عمليات URI قنوات تزويد بالبيانات والمعلومات السياقية الساكنة المحددة بعناوين (Resources) الموارد (RAG) التوليد المعزز بالاسترجاع قوالب موجهة جاهزة ومعدة مسبقاً لمساعدة المستخدم والنموذج على صياغة أفكارهما (Prompts) الموجهات المتكاملة لإنجاز المهام المطلوبة.

لا تقتصر تواصلية البروتوكول على استجابة الخادم لطلبات العميل، بل تمتد لتسمح للخادم بطلب خدمات متقدمة من العميل عبر قنوات مخصصة:

- آلية تتيح للخادم بدء طلبات تفاعلية عكسية مع النموذج اللغوي لإتمام عمليات تفكير (Sampling) أخذ العينات الإجرائية. متداخلة وتوليد محتوى جديد.
- تمكن الخادم من الاستعلام عن حدود بيئة العمل الحالية ومجلدات النظام المصرح (Roots) الاستعلام عن الجذور والمجالات له بالعمل ضمن نطاقها.
- تتيح للخادم طلب معلومات إضافية أو تأكيدات تكميلية من المستخدم لإنهاء (Elicitation) الاستنباط والطلب المباشر الإجراءات المعقدة.

المخصصة (stdio) آليتين للنقل؛ النقل المحلي عبر مجاري المدخلات والمخرجات القياسية MCP من الناحية الفنية، يدعم بروتوكول HTTP أو SSE والبروتوكولات القائمة على TLS-over-TCP لخوادم التشغيل المحلية، والنقل الشبكي المؤمن عبر اتصالات ولضمان عمل الأنظمة في بيئات الإنتاج الحقيقية، يدعم البروتوكول معايير أمان وتأمين شبكي صارمة؛ حيث يتعين تفعيل مصادقة لتجنب validateAuthorizationServerURL والتحقق الصارم من عناوين خوادم الترخيص باستخدام التابع البرمجي OAuth الصناعية وأدواتها المصنفة وظيفياً MCP خروقات سحب البيانات. يوضح الجدول التالي أمثلة حقيقية من خوادم

مخادمتها (MCP Server)	مخادمتها (MCP Server)	مخادمتها (MCP Server)
GitHub Official Server	إدارة المستودعات والتعليمات البرمجية (list_files, create_issue, open_pr, search_code)	استدعاء متبادل؛ يقرأ السياق المحلي ويقوم بعمليات التعديل في المستودع البعيد
Slack Platform Server	الاتصال وتنسيق قنوات التواصل الاجتماعي (send_[span_143](start_span)[s	دفع مستمر للمعلومات وتلقي الإشعارات دون تنفيذ أكواد داخلية

(MCP Server) الصناعي MCP خادم	(Exposed Tools & Functions) فئة ووظيفة الأدوات المتاحة	(Data Processing Pattern) آلية ونمط معالجة البيانات
	pan_143](end_span)message, list_channels, add_reaction)	
BigQuery SQL Toolbox	إدارة قواعد البيانات الضخمة وتحليل (execute_sql, [span_145](start_span)[span_145](end_span)list_dataset_ids, dry_run_cost)	(Raw stream) جلب وتحليل البيانات الصامت مع تصفية الاستعلامات وتحسين الأداء
Windows OS Server	التشغيل الآلي وأتمتة بيئة النظام (execute_command, execute_powershell, list_running_processes)	وصول بيئي مباشر وقوي يتطلب رقابة بشرية مشددة وآليات عزل أمنية
Figma Dev Mode Server	استخلاص الأنماط والرسوم المتجهية (get_frame_geometry, list[span_149](start_span)[span_149](end_span)_components, get_design_tokens)	قراءة وتصدير بيانات التصميم وتسليمها مباشرة لمستودع العميل البرمجي

أما فيما يتعلق بهيكل وتسمية الأدوات البرمجية، فيفرض البروتوكول قيوداً صارمة؛ إذ يجب أن يتراوح طول الاسم بين 1 و128 حرفاً، اللاتينية والأرقام والرموز المشتركة مثل الشرطة السفلية (_) والشرطة (-) والنقطة (.) دون فواصل ASCII وأن يقتصر على الأحرف لتحويل قيم المعاملات x-mcp-header أو مسافات. وتدعم وسيطات الأدوات تزيين الخصائص باستخدام علامة الوسم المخصصة مع التحذير من وضع كلمات المرور والرموز السرية بداخلها لانكشافها للوسطاء الشبكيين HTTP تلقائياً إلى ترويسات طلبات التحقق الصارم من تطابق هيكلية البيانات (outputSchema) ولتعزيز موثوقية الأنظمة الوكيل، يوفر توفير مخطط المخرجات الراجعة مع بنية لغات البرمجة المستهدفة لمنع تمرير بيانات تالفة إلى سياق النموذج اللغوي.

تحليل الأداء والتعقيد الحسابي للأدوات الإجرائية

إن استضافة الأدوات البرمجية داخل بيئات الذكاء الاصطناعي لا تخلو من تحديات تقنية وتربوية هامة؛ فمخططات الأدوات المكتوبة تُحقن مباشرة داخل نافذة السياق وتُعاد قراءتها ومعالجتها مع كل استدعاء للنموذج اللغوي، مما يسبب JSON Schema بصيغة وارتفاعاً ملحوظاً في فاتورة التشغيل وزمن الاستجابة. لحساب هذا التعقيد المالي والترمزي بدقة، (Tokens) استهلاكاً متزايداً للرموز لكل أداة. يمثل عبء المخططات الساكنة التي T_s من الأدوات المتاحة، بمتوسط حجم مخطط N نفترض أن النظام يحتوي على عدد يُرسل في كل طلب الصيغة:

$$T_{\{schemas\}} = \sum_{i=1}^N T_s(i)$$

من خطوات التكرار والملاحظة، وإرسال مخرجات الأدوات تبعاً لربط السياق M عند تشغيل الوكيل لحل مسألة برمجية معقدة تتطلب بعد إتمام المسيرة التكرارية يُعبر عنه بالمعادلة الحسابية التالية {total} التراكمي، فإن إجمالي الرموز المدخلة التراكمية

$$I_{\{total\}} = \sum_{j=1}^M \left(C_b + T_{\{schemas\}} + \sum_{k=1}^{j-1} (O_k + R_k) \right)$$

حجم مخرجات النموذج الفكرية والأوامر الموجهة للأدوات في O_k السياق والطلب الأولي للمستخدم، بينما يمثل C_b حيث يمثل حجم الاستجابة المرتجعة من تنفيذ الأداة في تلك الخطوة. يوضح هذا النموذج الرياضي بوضوح النمو R_k ويمثل k، خطوة التكرار التريبيعي لاستهلاك الرموز مع زيادة عدد الخطوات التكرارية، مما يفرض تبني آليات لتقليص الأثر المالي والزمني لعمليات الاستدعاء. ويوضح الجدول التالي تحليلاً تفصيلياً للاستهلاك والتعقيد المصاحب للأنماط المعمارية للوكلاء:

استراتيجيات تخفيف العبء المالي (Mitigation Tactics) والزمني	طبيعة استهلاك نافذة السياق (Context Token Profile)	التعقيد الحسابي والسياقي (Computational Complexity)	النمط المعماري للاستدعاء (Agent Calling Pattern)
تصفية المخططات واقتصارها على أدوات محددة بالطلب	منخفض ومستقر؛ يرسل الطلب مع المخطط وينتهي فوراً	خطوة واحدة مباشرة - O(1)	الاستدعاء الأحادي المباشر (Single-Step Call)
تحديد سقف تكرار صارم (MAX_ITERATIONS =)	تريبيعي تراكمي عالي جداً بسبب إعادة شحن تاريخ الفكر	ينمو خطياً مع عدد - O(M) الخطوات	التفكير والعمل المستمر (ReAct Loop)

النمط المعماري للاستدعاء (Agent Calling Pattern)	التعقيد الحسابي والسياقي (Computational Complexity)	طبيعة استهلاك نافذة السياق (Context Token Profile)	استراتيجيات تخفيف العبء المالي (Mitigation Tactics) والزماني
		والاستجابة	واقاطع الدائرة (10 \sim 15)
المخطط والمنفذ المستقل (Planner-Executor)	وضع خطة ثم - $O(P + E)$ تنفيذها تتابعياً	متوسط ومنضبط؛ يتم توليد الخطة كاملة ثم استدعاء المكونات	عزل مهام التخطيط في نموذج ذكي ومهام التنفيذ في نموذج أصغر وأقل تكلفة
الموجه الدائري الذكي (Router Pattern)	فترة واختيار مسبق - $O(1)$ للأدوات	خفيف ومتحكم به؛ يحقق فقط مخطط الأداة المرشحة	تطبيق نماذج تصنيف نية فائقة السرعة واختيار الأدوات بناءً على النية

لضمان سلامة النظم واستقرارها، تبرز أهمية تطبيق قيود التحكم والحماية البرمجية مثل وضع حد أقصى لمرات التكرار لمنع الدخول صارمة لا تتجاوز 30 ثانية في (Timeouts) في حلقات مفرغة عند فشل النماذج في معالجة المخرجات، وتفعيل مهلات زمنية التفاعلات اللحظية لمنع استهلاك الموارد المفرط.

الإجراءات العليا وتوليف النظم التوليدية الذكية

والبرمجة الفوقية قمة التجريد البرمجي الفعال؛ حيث تُعرف الإجراءات العليا بأنها (Meta-Procedures) تمثل الإجراءات العليا كيانات برمجية مصممة للوصول إلى بنية إجراءات أخرى وتعديلها ديناميكياً وتوليد مخططاتها وإثبات صحتها منطقياً قبل السماح بمرورها للبيئات التشغيلية. وتستلهم هذه الممارسة أصولها الأكاديمية من أبحاث جون شوت التأسيسية التي أعادت صياغة نماذج لغات "Lambda" كبدل لمنشئ لامبدا التقليدي "Vau" واقترح شوت استخدام المنشئ الإجرائي العملي Scheme و LISP (Applicative Evaluation)، بينما يعتمد منشئ لامبدا على تقييم المعاملات والوسيطات تلقائياً وضمنياً قبل دمجها وتمريضها للدالة (Operative Evaluation)، مما يمنح الإجراء، باستلام المعاملات في صورتها الخام غير المقيمة "Vau" يقوم المنشئ الإجرائي الفوقي السيطرة المطلقة لتحديد سياق وزمن وطريقة تقييم أو تعديل المعاملات والرموز المستلمة، فاتحاً آفاقاً واسعة للتحكم الفوقي بالعمليات البرمجية. المخصص لتوليد تفاعلات ومكالمات واعية VishBox v2 وتتجسد هذه القدرات العليا عملياً في أدوات محاكاة الجيل الثاني مثل إطار دلالي لمكافحة هجمات الهندسة الاجتماعية وعمليات الاحتيال الصوتي الموجهة. يعتمد الإطار على منهجية دمج كود إجرائي صارم لتوجيه سلوكيات المهاجمين الافتراضيين ورسم سيناريوهات (PPSE) ومبادئ الإقناع النفسي والاجتماعي (Procedural Code) يتولى فك رموز الاتصال وصياغة الخطط وتحديد سياق التنقل عبر (Main Agent) تفاعلية دقيقة، مستعيناً بوكيل رئيسي منسق الوحدات الفرعية بمرونة متكاملة.

مكتبة النماذج التطبيقية والنموذج التركيبي للتقييم

توفيراً لمكتبة أمثلة ونماذج مرجعية عملية جاهزة للتقييم والدمج البرمجي، يقدم هذا القسم نموذجاً برمجياً متكاملأً لأداة عليا تسمى يُعنى هذا الإجراء الفوقي باستقبال أهداف برمجية معقدة من وكيل (Procedure Orchestrator) المنسق الديناميكي للإجراءات مع صياغة إثبات (DAG) الذكاء الاصطناعي، ثم تحليل قائمة الأدوات البسيطة المتاحة لديه لبناء مسار تنفيذ مركب وخالي من الدورات رياضي لصحة هذا المسار مستنداً إلى قواعد منطق هور للتحقق الأكاديمي والتحليل التراجمي.

(JSON Schema) أولاً: مخطط تعريف الأداة العليا بتنسيق جيسون

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "name": "procedure_orchestrator",
  "description": "A meta-procedure that dynamically synthesizes a Directed Acyclic Graph (DAG) of executable procedure steps to achieve a complex, multi-layered programming goal, verified via Hoare-logic proofs.",
  "inputSchema": {
```

```

    "type": "object",
    "properties": {
      "goal_specification": {
        "type": "string",
        "description": "The high-level technical description of the
target task to be accomplished by synthesizing sub-procedures."
      },
      "available_procedures": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "name": {
              "type": "string",
              "pattern": "^[a-zA-Z0-9_.-]+$"
            },
            "description": { "type": "string" },
            "input_schema": { "type": "object" },
            "output_schema": { "type": "object" }
          },
          "required": ["name", "input_schema"]
        },
        "description": "List of primitive operations currently exposed
by active MCP servers."
      },
      "strict_execution": {
        "type": "boolean",
        "default": true,
        "description": "If true, enforces rigorous type validation and
transactional rollback on step failure."
      }
    },
    "required": ["goal_specification", "available_procedures"],
    "additionalProperties": false
  },
  "outputSchema": {
    "type": "object",
    "properties": {
      "execution_graph": {
        "type": "object",
        "properties": {
          "steps": {
            "type": "array",
            "items": {
              "type": "object",
              "properties": {
                "step_id": { "type": "integer" },
                "procedure_name": { "type": "string" },

```

```

        "arguments_mapping": { "type": "object" },
        "dependencies": {
            "type": "array",
            "items": { "type": "integer" }
        }
    },
    "required": ["step_id", "procedure_name",
"arguments_mapping", "dependencies"]
    }
},
    "required": ["steps"]
},
    "verification_proof": {
        "type": "string",
        "description": "A Hoare-logic triple verification outline
certifying the logical correctness of the synthesized execution
graph."
    }
},
    "required": ["execution_graph", "verification_proof"],
    "additionalProperties": false
}
}

```

ثانياً: سيناريو تطبيقي عملي (Sample Execution Run Payload)

يمثل كود جيسون التالي حمولة طلب الاستدعاء الفعلي المرسل إلى خادم الأداة العليا لتكليفها بتوليف مخطط لعملية معالجة ملفات مخصصة ونقلها:

```

{
    "jsonrpc": "2.0",
    "id": "req_99824",
    "method": "tools/call",
    "params": {
        "name": "procedure_orchestrator",
        "arguments": {
            "goal_specification": "Compress local database dumps and upload
them secure-file transfer protocol, trigger email alert if upload
fails.",
            "available_procedures": [
                {
                    "name": "compress_data_gzip",
                    "description": "Compress raw files to archive storage.",
                    "input_schema": {
                        "type": "object",
                        "properties": { "source_file_path": { "type": "string" }
                    }
                }
            ]
        }
    }
},

```



```

        "required": ["source_file_path"]
    },
    {
        "name": "sftp_upload",
        "description": "Upload archive to remote secure hosting.",
        "input_schema": {
            "type": "object",
            "properties": {
                "archive_path": { "type": "string" },
                "remote_host": { "type": "string" }
            },
            "required": ["archive_path", "remote_host"]
        }
    },
    {
        "name": "send_email_notification",
        "description": "Sends execution notification to
administrator.",
        "input_schema": {
            "type": "object",
            "properties": {
                "recipient": { "type": "string" },
                "body": { "type": "string" }
            },
            "required": ["recipient", "body"]
        }
    }
],
"strict_execution": true
}
}
}

```

تُنتج الأداة العليا استجابة مهيكلة ومصدقة تحتوي على رسم بياني منظم لإنجاز المهمة المطلوبة بدقة متكاملة

```

{
    "jsonrpc": "2.0",
    "id": "req_99824",
    "result": {
        "resultType": "object",
        "execution_graph": {
            "steps": [
                {
                    "step_id": 1,
                    "procedure_name": "compress_data_gzip",
                    "arguments_mapping": { "source_file_path":
"/var/dumps/db_today.sql" },
                    "dependencies": []
                }
            ]
        }
    }
}

```

```

    },
    {
      "step_id": 2,
      "procedure_name": "sftp_upload",
      "arguments_mapping": {
        "archive_path": "$steps[1].output.compressed_path",
        "remote_host": "sftp.backup.enterprise.com"
      },
      "dependencies": [1]
    },
    {
      "step_id": 3,
      "procedure_name": "send_email_notification",
      "arguments_mapping": {
        "recipient": "security_ops@enterprise.com",
        "body": "System run alert. Database compressed and
uploaded to secure remote destination. Hoare transition certified."
      },
      "dependencies": [2]
    }
  ]
},
"verification_proof": "Verified by establishing Hoare triple:
{Pre: File_Exists(/var/dumps/db_today.sql)} Steps(1;2;3) {Post:
Upload_Success == True && Notify_Sent == True}. Executed via backward
verification rules."
}
}

```

لتوفير أداة هندسية موضوعية لتقييم مدى قابلية استقرار وتطبيق الإجراءات البرمجية والأدوات المستضافة داخل أنظمة الذكاء (Procedure Intelligence) الاصطناعي، يطرح هذا الإطار نموذجاً قياسياً موحداً يُعرف باسم "مؤشر حيوية ذكاء الإجراء Viability Score - PIVS):

$$PIVS = \frac{\alpha \cdot \text{Asci} + \beta \cdot \text{Comp}}{\gamma \cdot \text{Overhead} + \delta \cdot \text{ErrorRate}}$$

تمثل مكونات هذا النموذج الرياضي مجموعة من المقاييس الفنية الصارمة القابلة للاختبار الكمي:

- **مؤشر رقمي من 1 إلى 10** يقيس جودة مخطط المدخلات والمخرجات (**Asci**): **وضوح الإثبات والتحقق الحصري** للأداة وخطو تسميتها وصياغتها من اللبس الدلالي ومطابقتها لمبادئ بيدفورد.
- **مؤشر كمي يقيس مدى اندماج الأداة البرمجية مع مكتبة العمليات** (**Comp**): **تكامل الأنطولوجيا وتغطيتها المعرفية** وقدرتها على الارتباط بأنشطة حقيقية مصدقة (SP-OPL) المعيارية بالمؤسسة.
- **حجم استهلاك المخطط الهيكلي للأداة داخل نافذة السياق** مقيساً بالآلاف (**Overhead**): **عبء حجم سياق الأداة** حيث تشير القيم المنخفضة إلى كفاءة التصميم الهيكلي، (K-Tokens) الرموز.
- **النسبة المئوية للأخطاء والتعثر أثناء التشغيل الفعلي للأداة** (**ErrorRate**): **معدل فشل التشغيل العملي** (Tool Execution Errors) نتيجة خلل في جودة المدخلات أو الاتصال.
- **المعاملات النسبية لموازنة الأوزان** ($\alpha, \beta, \gamma, \delta$): **معاملات موجبة يحددها مهندسو النظام لموازنة وتعديل كفة المؤشر وفق أولويات بيئة العمل** (على سبيل المثال، عند بناء أنظمة منخفضة التكلفة، تُرفع قيمة المعامل الحاكم لعامل السياق).

الاستنتاجات والتوصيات الهندسية لمطوري الأنظمة الوكيلية

يفضي التحليل المتكامل للإجراءات البرمجية وتفاعلها في بيئات الذكاء الاصطناعي إلى صياغة مجموعة من التوصيات الفنية والخطوات الهندسية اللازمة لبناء أنظمة وكيلية آمنة وفعالة:

- **التصميم المحكم للمخططات وتصفية السياق:** يتعين على المطورين تجنب حشو البيانات الوكيلية بمخططات أدوات غير لتصنيف نية المستخدم أولاً، ثم حقن مخططات الأدوات (Router Pattern) ضرورية. ويوصى بتبني نمط الموجه الذكي المطلوبة حصرياً لتقليل النمو التريبيعي لاستهلاك الرموز وتقادي تجاوز حدود نافذة السياق.
- **تفعيل وضعيات التحقق الصارم والمخرجات المهيكلية:** لحماية الأنظمة البرمجية من تمرير وسيطات ملوثة أو تالفة، يُنصح ومطالبة الخوادم بتسليم مخطط استجابة مصدق (strict: true) بتفعيل التثبيت الصارم للمخططات في تعريف الأدوات لإجراء مطابقة دلالية شاملة قبل تمرير البيانات للأنظمة الداخلية (outputSchema).
- **على الأدوات التي تعمل في stdin/stdout تطبيق العزل الشبكي وإجراءات الأمان المتقدمة:** يجب قصر النقل المحلي عبر OAuth نطاق أمن ومعزول محلياً، مع إلزام الاتصالات البعيدة بتبني بروتوكولات آمنة والتحقق من يوابات الهوية ومصادر الموثوقة لمنع تسريب البيانات الحساسة.
- **دمج المراقبة البشرية في العمليات ذات الآثار الجانبية:** بالنسبة للإجراءات البرمجية التي تنطوي على آثار جانبية ملموسة وغير متماثلة (مثل العمليات المالية وتعديل قواعد البيانات الحية)، يجب إلزام الأنظمة بطلب تأكيد بشري صريح وإظهار مؤشرات مرئية قبل تفعيل الخطوة لضمان بقاء المستخدم البشري متحكماً بمسارات العمل البرمجية والتحويلات الأمنية الحساسة.

Works cited

1. SIR MAURICE VINCENT WILKES - Royal Society Publishing, <https://royalsocietypublishing.org/doi/pdf/10.1098/rsbm.2013.0020>
2. The EDSAC and Computing in Cambridge | Whipple Museum of the History of Science, <https://www.whipplemuseum.cam.ac.uk/explore-whipple-collections/calculating-devices/edsac-and-computing-cambridge>
3. On This Day - EDSAC's First Run - I Programmer, <https://www.i-programmer.info/news/82-heritage/5839-on-this-day-edsacs-first-run.html>
4. David J Wheeler - Computer Pioneers, <https://history.computer.org/pioneers/wheeler.html>
5. EDSAC - Wikipedia, <https://en.wikipedia.org/wiki/EDSAC>
6. Wheeler Jump - Grokipedia, https://grokipedia.com/page/wheeler_jump
7. David Wheeler (computer scientist) - Wikipedia, [https://en.wikipedia.org/wiki/David_Wheeler_\(computer_scientist\)](https://en.wikipedia.org/wiki/David_Wheeler_(computer_scientist))
8. David Wheeler (1927–2004) | The World of SPlpectrum, <https://splpectrum.world/positioning/persons/w/wheeler-d/>
9. What was the first introduction of callback concept of programming?, <https://retrocomputing.stackexchange.com/questions/13479/what-was-the-first-introduction-of-callback-concept-of-programming>
10. The Preparation of Programs for an Electronic Digital Computer - Wikipedia, https://en.wikipedia.org/wiki/The_Preparation_of_Programs_for_an_Electronic_Digital_Computer
11. Introduction to the Lambda calculus - FutureLearn, <https://www.futurelearn.com/info/courses/functional-programming-haskell/0/steps/27249>
12. lambda-calculus - It's Actors All The Way Down, <https://dalnefre.com/wp/tag/lambda-calculus/>
13. Alonzo Church and Lambda Calculus: Foundations of Computing - Kronecker Wallis, <https://www.kroneckerwallis.com/alonzo-church-and-lambda-calculus-foundations-of-computing/>
14. Why is lambda calculus so "function" oriented?, <https://cstheory.stackexchange.com/questions/30546/why-is-lambda-calculus-so-function-oriented>
15. Function vs Procedure - GeeksforGeeks, <https://www.geeksforgeeks.org/computer-science-fundamentals/difference-between-function-and-procedure/>
16. What is the difference between a "function" and a "procedure"? - Stack

Overflow,

<https://stackoverflow.com/questions/721090/what-is-the-difference-between-a-function-and-a-procedure> 17. Function (computer programming) - Wikipedia,

[https://en.wikipedia.org/wiki/Function_\(computer_programming\)](https://en.wikipedia.org/wiki/Function_(computer_programming)) 18. Can you explain the difference between a function and a procedure in programming languages such as Java, Python, C++, and OCaml? - Quora,

<https://www.quora.com/Can-you-explain-the-difference-between-a-function-and-a-procedure-in-programming-languages-such-as-Java-Python-C-and-OCaml> 19. Software Specification: Formal Reasoning with Hoare Logic,

<https://web.tecnico.ulisboa.pt/~joaofernandoferreira/2021/SoftSpec/02-hoare.html> 20. Hoare Logic: A Thorough Guide to Correctness, Reasoning, and Practical Applications,

<https://dreforrester.co.uk/hoare-logic/> 21. Hoare Logic and Model Checking - University of Cambridge, <https://www.cl.cam.ac.uk/teaching/1617/HLog+ModC/slides/part1.pdf> 22. Hoare logic - Wikipedia, https://en.wikipedia.org/wiki/Hoare_logic 23. Forward with Hoare | KTH,

<https://www.kth.se/social/files/5901f355f276545e53285a96/forward-hoare.pdf> 24. A Taxonomy for Design Decisions in Software Architecture Documentation - KIT,

<https://publikationen.bibliothek.kit.edu/1000160794/152927102> 25. software development tools: a reference guide to a taxonomy of tool features - GovInfo,

<https://www.govinfo.gov/content/pkg/GOVPUB-C13-c03c0a579399b35c0b758d660d5a190a/pdf/GOVPUB-C13-c03c0a579399b35c0b758d660d5a190a.pdf> 26. Optimization of an EHR Mobile Application Using the UfuRT Conceptual Framework - PMC,

<https://pmc.ncbi.nlm.nih.gov/articles/PMC3540562/> 27. A software process ontology as a common vocabulary about software processes Ricardo de Almeida Falbo* and Gleidson Bertollo - Nemo,

https://nemo.inf.ufes.br/wp-content/papercite-data/pdf/a_software_process_ontology_as_a_common_vocabulary_about_software_processes_2009.pdf 28. Using Ontology Patterns for Building a Reference Software Testing Ontology - ResearchGate,

https://www.researchgate.net/publication/261450604_Using_Ontology_Patterns_for_Building_a_Reference_Software_Testing_Ontology 29. Using Ontology Patterns for Building a Reference Software Testing Ontology,

<http://mtc-m21b.sid.inpe.br/attachment.cgi/sid.inpe.br/mtc-m21b/2014/01.21.15.32/doc/Using%20Ontology%20Patterns%20for%20Building%20a%20Reference%20Software%202013.pdf> 30. Building AI Agents from Scratch: A Deep Dive into Function Calling, Tool Use, and Agentic Patterns - DEV Community,

https://dev.to/pockit_tools/building-ai-agents-from-scratch-a-deep-dive-into-function-calling-tool-use-and-agentic-patterns-382g 31. Model Context Protocol (MCP) explained: A practical technical overview for developers and architects - CodiLime,

<https://codilime.com/blog/model-context-protocol-explained/> 32. Specification - Model Context Protocol, <https://modelcontextprotocol.io/specification/2025-11-25> 33. What is Model Context Protocol (MCP) and what makes it useful? | Parallel Web Systems,

<https://parallel.ai/articles/what-is-mcp> 34. Overview - Model Context Protocol, <https://modelcontextprotocol.io/specification/draft/basic> 35. Tools - Model Context Protocol, <https://modelcontextprotocol.io/specification/draft/server/tools> 36. Resources - Model Context Protocol (MCP), <https://modelcontextprotocol.info/specification/2024-11-05/server/resources/> 37. What is MCP (Model Context Protocol)? A Developer's Guide - Encore Cloud,

<https://encore.dev/articles/what-is-mcp> 38. Model Context Protocol (MCP) - AI SDK, <https://ai-sdk.dev/docs/ai-sdk-core/mcp-tools> 39. Model Context Protocol explained as simply as possible - Sean Goedecke, <https://www.seangoedecke.com/model-context-protocol/> 40. Model

Context Protocol (MCP) Extensions for Network Equipment Management - IETF, <https://www.ietf.org/archive/id/draft-zw-opsawg-mcp-network-mgmt-00.html> 41. Function Calling in AI Agents - Prompt Engineering Guide, <https://www.promptingguide.ai/agents/function-calling> 42. Function calling et tool use : la mécanique des agents IA - Proactive Academy, <https://proactiveacademy.fr/blog/agentsia/function-calling-tool-use/> 43. Function Calling vs Tool Use in AI Agents - Propelius Technologies, <https://propelius.ai/blogs/function-calling-vs-tool-use-ai-agents/> 44. Tool-Calling Architecture Patterns for AI Agents - Medium, <https://medium.com/@vasanthancomrads/tool-calling-architecture-patterns-for-ai-agents-91c82333d662> 45. VishBox v2: A Multi-Agent System for Adaptive Voice Phishing Simulation - ACL Anthology, <https://aclanthology.org/2026.acl-industry.145.pdf>